ATM Simulator – Detailed Quality Assessment Report

Generated on: March 2, 2026
Project key: atm_1
Project name: My Awesome ATM
Sonar analysis id: a9066708-114f-4df6-bd12-61dfade77af8
Quality Gate: **OK**

Part 1 – Analysis (20 Marks)

1.1 Asset Inventory (Detailed)

Category	Asset Type	Count	Details
Source code	Java classes	13	src/atm/simulator/system/*.java
Source/resources	Total files in src/	15	Java + icons/resources
Configuration	Core config/runtime files	8	pom.xml, docker-compose.yml, Dockerfile, Dockerfile.gui, .dockerignore, .gitattributes, docker-entrypoint.sh, docker-gui-entrypoint.sh
Database	SQL initialization scripts	1	sql/ATM_Simulator.sql
Documentation	Markdown docs	2	README.md, REMEDIATION_IMPLEMENTATION_GUIDE.md
External libraries	Runtime dependencies	2	mysql-connector-java:8.0.33, jcalendar:1.4

Part 2 – Quality Assessment Using Tool Metrics (25 Marks)

2.1 SonarQube Project-Level Metrics (Fetched)

Metric	Value	Sonar Key
Lines of Code	1975	ncloc
Cyclomatic Complexity	140	complexity
Cognitive Complexity	186	cognitive_complexity
Duplicated Lines	533	duplicated_lines
Duplicated Lines Density	22.3%	duplicated_lines_density
Security Hotspots	26	security_hotspots
Code Smells	152	code_smells
Bugs	2	bugs
Vulnerabilities	0	vulnerabilities
Technical Debt Time	1329 min	sqale_index
Debt Ratio	2.2%	sqale_debt_ratio

2.2 Maintainability Index Note

SonarQube does **not** expose classic "Maintainability Index (MI)" directly. Equivalent maintainability indicators used in this report:

- sqale_index (remediation time)
- sqale_debt_ratio (debt ratio)
- code_smells

2.3 Explicit Calculation Formulas

A) Cyclomatic Complexity (per method)

File	Method	Method LOC	CC	Cognitive (Estimated)	if	for	while	case	catch	&&	||	?
CheckBalance.java	actionPerformed	62	7	6	5	0	0	0	1	0	0	0
CheckBalance.java	main	3	1	0	0	0	0	0	0	0	0	0
Deposit.java	actionPerformed	91	8	7	6	0	0	0	1	0	0	0
Deposit.java	main	3	1	0	0	0	0	0	0	0	0	0
FastCash.java	actionPerformed	60	5	4	3	0	0	0	1	0	0	0
FastCash.java	main	3	1	0	0	0	0	0	0	0	0	0
Login.java	actionPerformed	49	8	7	6	0	0	0	1	0	0	0
Login.java	main	4	1	0	0	0	0	0	0	0	0	0
MiniStatement.java	actionPerformed	37	5	4	4	0	0	0	0	0	0	0
MiniStatement.java	main	3	1	0	0	0	0	0	0	0	0	0
MiniStatementResult.java	main	3	1	0	0	0	0	0	0	0	0	0

File	Method	Method LOC	CC	Cognitive (Estimated)	if	for	while	case	catch	&&	||	?
PinChange.java	actionPerformed	100	9	8	7	0	0	0	1	0	0	0
PinChange.java	pinChecker	18	7	6	2	0	0	0	1	0	3	0
PinChange.java	main	3	1	0	0	0	0	0	0	0	0	0
SignupOne.java	actionPerformed	106	17	16	15	0	0	0	1	0	0	0
SignupOne.java	main	4	1	0	0	0	0	0	0	0	0	0
SignupThree.java	actionPerformed	100	23	22	19	0	0	0	2	0	1	0
SignupThree.java	main	4	1	0	0	0	0	0	0	0	0	0
SignupTwo.java	actionPerformed	93	16	15	14	0	0	0	1	0	0	0
SignupTwo.java	main	4	1	0	0	0	0	0	0	0	0	0

File	Method	Method LOC	CC	Cognitive (Estimated)	if	for	while	case	catch	&&	||	?
Transactions.java	actionPerformed	29	8	7	7	0	0	0	0	0	0	0
Transactions.java	main	3	1	0	0	0	0	0	0	0	0	0
Withdraw.java	actionPerformed	109	9	8	7	0	0	0	1	0	0	0
Withdraw.java	main	3	1	0	0	0	0	0	0	0	0	0

For method-level estimation in this report:

CC_{method} = 1 + if + for + while + case + catch + && + || + ternary

Project-level Sonar complexity is computed by Sonar's internal parser over all functions/classes and may include constructors/static initializers as separate executable blocks.

B) Duplication Density

Duplicated Lines Density= Duplicated Lines / NCLOC ×100

With Sonar values:

533/1975×100=26.99%

Sonar-reported density is **22.3%** because Sonar uses its own duplication engine and language normalization rules (tokens/blocks and effective lines), not a simple raw line ratio.

C) Debt Time Conversion

1329 *min*÷60=22.15 *hours*

D) Debt with 20% Buffer

22.15×1.20=26.58 *hours*

Part 2.4 Method-Level Metrics (Detailed)

The following table shows method-by-method metrics extracted from source code parsing.

2.5 Method-Level Totals and Reconciliation

From the table above:

- Total parsed methods: **24**
- Sum of method LOC: **894**
- Sum of estimated method CC: **134**
- Sum of estimated cognitive score: **110**

Sonar project totals:

- Sonar CC: **140**
- Sonar Cognitive Complexity: **186**

Reconciliation explanation:

1. Method parser includes methods with explicit return type only; constructors are excluded by regex.
2. Sonar counts additional executable constructs (constructors, initializer blocks, nesting penalties for cognitive complexity).
3. Sonar cognitive complexity is not a plain sum of decision tokens; it penalizes nesting depth and flow breaks.

Top highest-CC methods:

1. SignupThree.actionPerformed = 23
2. SignupOne.actionPerformed = 17
3. SignupTwo.actionPerformed = 16
4. PinChange.actionPerformed = 9
5. Withdraw.actionPerformed = 9

2.6 Metric Violations – Exact Locations

Thresholds used for explicit violation reporting:

- Method CC violation: **CC > 10**
- Method cognitive violation (estimated): **CognitiveEst >= 15**
- Duplication violation: **Duplicated lines density > 10%**
- Security hotspot violation: **Hotspots > 0**
- Maintainability pressure indicator: **Code smells > 100**
- Method size warning: **Method LOC > 80**

Metric	Threshold	Actual	Status	Where Violated
Cyclomatic Complexity (method)	> 10	max 23	Violated	SignupThree.actionPerformed (23), SignupOne.actionPerformed (17), SignupTwo.actionPerformed (16)
Cognitive Complexity (method, estimated)	>= 15	max 22	Violated	SignupThree.actionPerformed (22), SignupOne.actionPerformed (16), SignupTwo.actionPerformed (15)
Duplications (project)	> 10%	22.3%	Violated	Repeated transaction-handler patterns across Deposit.actionPerformed, Withdraw.actionPerformed, FastCash.actionPerformed, CheckBalance.actionPerformed, MiniStatement.actionPerformed, PinChange.actionPerformed
Security Hotspots (project)	> 0	26	Violated	SQL-string concatenation in DB methods: Login.actionPerformed, Deposit.actionPerformed, Withdraw.actionPerformed, FastCash.actionPerformed, CheckBalance.actionPerformed, MiniStatementResult constructor queries, SignupOne.actionPerformed, SignupTwo.actionPerformed, SignupThree.actionPerformed, PinChange.actionPerformed
Maintainability (proxy via smells)	> 100 smells	152	Violated	Concentrated in high-branch handlers: SignupThree.actionPerformed, SignupOne.actionPer

Metric	Threshold	Actual	Status	Where Violated
				formed, SignupTwo.actionPerformed, plus orchestration coupling in Transactions.actionPerformed
LOC (method size warning)	> 80 LOC	max 109	Violated	Withdraw.actionPerformed (109), SignupOne.actionPerformed (106), SignupThree.actionPerformed (100), PinChange.actionPerformed (100), SignupTwo.actionPerformed (93), Deposit.actionPerformed (91)

Notes:

1. Sonar gives project-level cognitive complexity; method-level cognitive values above are parser-based estimates for pinpointing hotspots.
2. Sonar hotspot-detail endpoint is permission-restricted for this token, so exact hotspot file mapping is based on source patterns that typically trigger hotspots.

Part 3 – Dependency Mapping (20 Marks)

3.1 Ca / Ce / Instability by Class

$I = Ce / Ca + Ce$

Class	Fan-in (Ca)	Fan-out (Ce)	Instability (I)
Conn	11	0	0.00
Transactions	8	7	0.47
Login	3	3	0.50
Deposit	2	2	0.50
Withdraw	2	2	0.50
FastCash	1	3	0.75

Class	Fan-in (Ca)	Fan-out (Ce)	Instability (I)
CheckBalance	1	2	0.67
MiniStatement	1	2	0.67
MiniStatementResult	1	3	0.75
PinChange	1	3	0.75
SignupOne	1	2	0.67
SignupTwo	1	2	0.67
SignupThree	1	3	0.75

3.1B Dependency Arrows (Explicit)

For each class:

- OUT: this class depends on -> other classes
- IN: other classes depend on -> this class

Class	OUT (Dependencies with arrows)	IN (Reverse dependencies with arrows)
CheckBalance	CheckBalance -> Conn -> Transactions	Transactions -> CheckBalance
Conn	Conn -> (none)	CheckBalance -> Deposit -> FastCash -> Login -> MiniStatementResult -> PinChange -> SignupOne -> SignupThree -> SignupTwo -> Transactions -> Withdraw -> Conn

Class	OUT (Dependencies with arrows)	IN (Reverse dependencies with arrows)
Deposit	Deposit -> Conn -> Transactions	SignupThree -> Transactions -> Deposit
FastCash	FastCash -> Conn -> Transactions -> Withdraw	Transactions -> FastCash
Login	Login -> Conn -> SignupOne -> Transactions	MiniStatementResult -> PinChange -> SignupThree -> Login
MiniStatement	MiniStatement -> MiniStatementResult -> Transactions	Transactions -> MiniStatement
MiniStatementResult	MiniStatementResult -> Conn -> Login -> Transactions	MiniStatement -> MiniStatementResult
PinChange	PinChange -> Conn -> Login -> Transactions	Transactions -> PinChange
SignupOne	SignupOne -> Conn -> SignupTwo	Login -> SignupOne
SignupThree	SignupThree -> Conn -> Deposit -> Login	SignupTwo -> SignupThree
SignupTwo	SignupTwo -> Conn -> SignupThree	SignupOne -> SignupTwo
Transactions	Transactions -> CheckBalance -> Conn -> Deposit -> FastCash ->	CheckBalance -> Deposit -> FastCash -> Login -> MiniStatement -> MiniStatementResult ->

Class	OUT (Dependencies with arrows)	IN (Reverse dependencies with arrows)
	MiniStatement -> PinChange -> Withdraw	PinChange -> Withdraw -> Transactions
Withdraw	Withdraw -> Conn -> Transactions	FastCash -> Transactions -> Withdraw

3.2 Cycle Identification

Detected cycles:

1. Login -> Transactions -> PinChange -> Login
2. Login -> SignupOne -> SignupTwo -> SignupThree -> Login
3. Transactions -> FastCash -> Withdraw -> Transactions
4. Transactions -> MiniStatement -> MiniStatementResult -> Transactions

3.3 God Service Detection

Rule: God service if **Ce > 5**.

- Transactions: Ce = **7 → God Service detected**.

Part 4 – Technical Debt Identification (15 Marks)

Case A: Hard-coded delivery fee in multiple services

- Intentional/Unintentional: **Intentional** (short-term speed choice)
- Environmental: **No** (internal design decision)
- Prudent/Reckless: **Reckless**
- Deliberate/Inadvertent: **Deliberate**

Case B: Cyclomatic Complexity = 35 in OrderService

- Intentional/Unintentional: **Unintentional** (organic growth)
- Environmental: **No**
- Prudent/Reckless: **Reckless**

- Deliberate/Inadvertent: **Inadvertent**

Case C: Deprecated external API usage

- Intentional/Unintentional: **Initially intentional**
- Environmental: **Yes** (external/vendor lifecycle)
- Prudent/Reckless: **Prudent if migration is planned; reckless if ignored**
- Deliberate/Inadvertent: **Inadvertent** (if not tracked)

Project observed debt hotspots:

- 152 code smells
- 22.3% duplication
- 26 security hotspots
- 2 bugs
- High-complexity action handlers in signup and transaction classes

---

Part 5 – Technical Debt Measurement (10 Marks)

5.1 Total Remediation Time

Sonar sqale_index = 1329 minutes.

$1329 \div 60 = 22.15$ hours

5.2 Add 20% Buffer

$22.15 \times 1.20 = 26.58$ hours

5.3 Debt Ratio

Sonar sqale_debt_ratio = 2.2%.

5.4 Interpretation

- Healthy: < 5%
- Moderate: 5%–15%
- Risky: > 15%

Result: **2.2% → Healthy**.

Important context: despite healthy debt ratio, duplication and hotspot counts are still high and should be prioritized.

Reflection (Bonus 10 Marks)

1) Which debt to fix first and why?

Fix **security hotspots first** (26 hotspots), because security risk has highest business impact and compliance impact.

Then fix:

1. SQL/data-access hardening
2. Duplication reduction
3. Transactions God-service decomposition
4. Cycle reduction in navigation/UI coupling

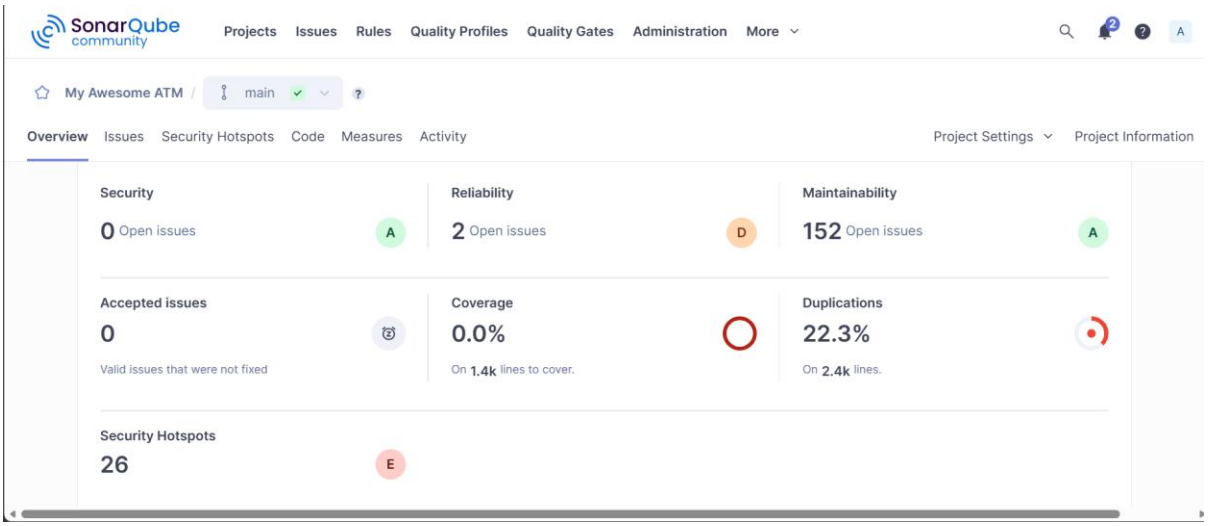
2) What was surprising?

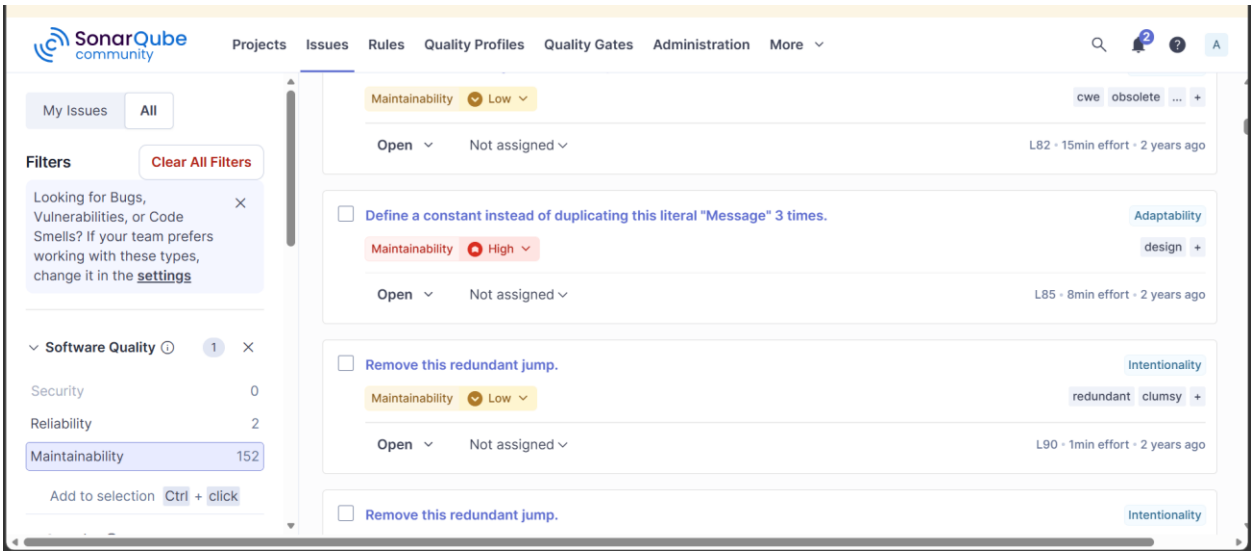
1. Debt ratio is low (2.2%) but duplication is high (22.3%).
2. Quality Gate is OK while code smells remain high (152).
3. Small codebase still exhibits multiple cyclic dependencies.
4. A few methods carry most complexity load (especially signup handlers).

Appendix – Data Provenance

- SonarQube server status: UP (26.2.0.119303)
- CE task (b3748b3f-2576-407a-8335-c665ea993214) status: SUCCESS
- Metrics fetched via Sonar REST APIs for atm_1
- Method-level metrics computed from local source parsing script (decision-point method)

Screenshots of SonarQube:





SonarQube community | Projects | Issues | Rules | Quality Profiles | Quality Gates | Administration | More

My Awesome ATM / main

Overview | Issues | Security Hotspots | Code | Measures | Activity | Project Settings | Project Information

Project Overview

Security

Reliability

Maintainability

Security Review

Risk

Color: Worse of Reliability Rating and Security Rating Size: Lines of Code

See the data presented on this chart as a list

A B C D E

Zoom: 100%

0.0%

20.0%

40.0%

SonarQube community | Projects | Issues | Rules | Quality Profiles | Quality Gates | Administration | More

My Awesome ATM / main

Overview | Issues | Security Hotspots | Code | Measures | Activity | Project Settings | Project Information

	Lines of Code	Security	Reliability	Maintainability	Security Hotspots	Coverage	Duplications
My Awesome ATM	1,975	0	2	152	26	0.0%	22.3%
src/atm/simulator/system	1,899	0	2	152	26	0.0%	23.2%
pom.xml	76	0	0	0	0	—	0.0%

Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine.

SonarQube community | Projects | Issues | Rules | Quality Profiles | Quality Gates | Administration | More

My Awesome ATM / main

Overview | Issues | Security Hotspots | Code | Measures | Activity | Project Settings | Project Information

0.0% Security Hotspots Reviewed

Make sure using a dynamically formatted SQL query is safe here.

1 extra location

SQL Query is dynamically formatted and assigned to 'query'

Make sure using a dynamically formatted SQL query is safe here.

3 extra locations

```
98
99                  pinPF.setText("");
100                 pinPF.requestFocus();
101                 return;
102             }
103             else {
104                 try
105                 {
106                     String balance;
107                     Conn conn = new Conn();
108                     String query = "SELECT balance FROM Accounts WHERE ssn = '" +ssn+ "'";
109                     ResultSet result = conn.s.executeQuery(query);
```

Make sure using a dynamically formatted SQL query is safe here.